

Modelos Generativos Profundos

Clase 9: Modelos autorregresivos y LLMs (parte V)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Pre-training y optimización

Large language models

Alignment y fine-tuning

Pre-training y optimización

El entrenamiento de un LLM moderno es un proceso complejo que involucra varias técnicas y heurísticas para optimizar el rendimiento y la eficiencia.

En particular, el alto costo computacional y la inestabilidad del entrenamiento requieren estrategias específicas para manejar grandes modelos y datasets masivos.

Entrenamiento distribuido

Entrenar un LLM moderno tiene varias dificultades:

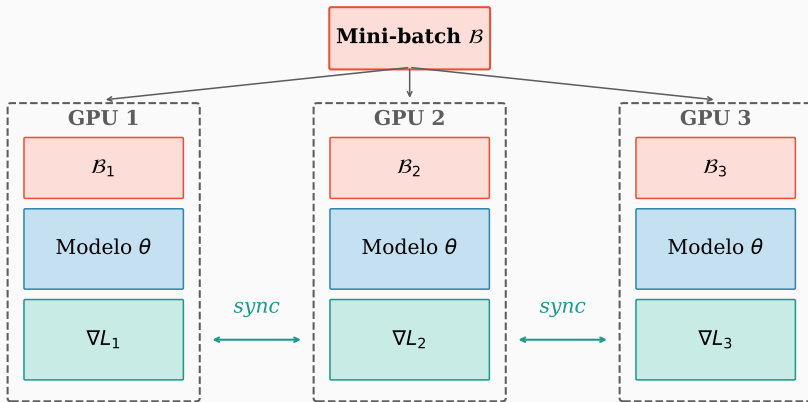
- El modelo no cabe en la memoria de una sola GPU.
- El tiempo de entrenamiento sería prohibitivo en una sola GPU.
- Los datasets de pre-training contienen billones de tokens.

Debido a esto, es usual distribuir el entrenamiento en múltiples GPUs. Existen distintas estrategias:

- **Paralelización de datos.**
- **Paralelización de modelo**

Paralelización de datos

En **data parallelism**, el modelo se replica en cada GPU y los datos se dividen en mini-batches. Cada GPU calcula gradientes locales que luego se agregan mediante **all-reduce**.

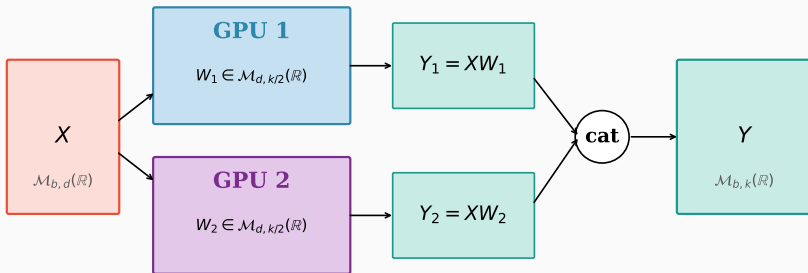


$$\nabla_{\theta} L = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L(B_i)$$

Paralelización de tensor

En **tensor parallelism** (tipo de model parallelism), las operaciones matriciales se dividen entre GPUs.

Luego, las salidas parciales se combinan (e.g. mediante all-reduce) para el siguiente paso.



$$Y = XW = X[W_1|W_2] = [XW_1|XW_2] = [Y_1|Y_2]$$

Learning rate scheduling

Una técnica crucial para el entrenamiento de LLMs es el **learning rate scheduling**, donde la tasa de aprendizaje $\eta_t > 0$ varía durante el entrenamiento. Algunas técnicas usuales son:

- **Decaimiento exponencial:** $\eta_t = \eta_0 \cdot \gamma^t$, con $\gamma < 1$.
- **Cosine annealing:** decae suavemente de η_0 a $\eta_{\min}$ en T pasos:

$$\eta_t = \eta_{\min} + \frac{\eta_0 - \eta_{\min}}{2} \left(1 + \cos \left(\frac{\pi t}{T} \right) \right)$$

- **Step decay:** reduce η_t por un factor fijo cada k épocas.

Warmup

Al inicio del entrenamiento:

- Los pesos están inicializados aleatoriamente, produciendo gradientes ruidosos.
- Las estadísticas de Adam aún no han convergido.
- Una tasa de aprendizaje alta puede causar actualizaciones muy grandes, llevando a divergencia o inestabilidad numérica.

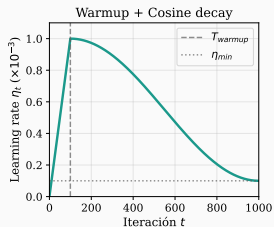
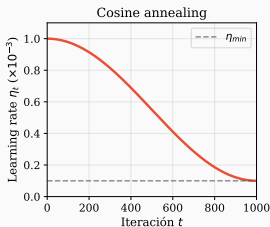
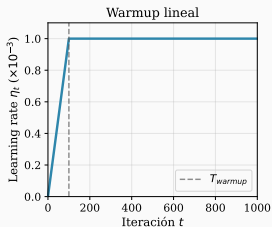
Por lo tanto, es usual usar un **learning rate warmup** al inicio del entrenamiento, incrementando gradualmente la tasa de aprendizaje durante las primeras T_{warmup} iteraciones:

$$\eta_t = \eta_0 \cdot \frac{t}{T_{\text{warmup}}}, \quad \text{para } t \leq T_{\text{warmup}}$$

Warmup + cosine decay

El esquema **warmup + cosine decay** es estándar en LLMs:

$$\eta_t = \begin{cases} \eta_0 \cdot \frac{t}{T_{\text{warmup}}} & \text{si } t \leq T_{\text{warmup}} \\ \eta_{\min} + \frac{\eta_0 - \eta_{\min}}{2} \left(1 + \cos \left(\frac{\pi(t - T_{\text{warmup}})}{T - T_{\text{warmup}}} \right) \right) & \text{si no} \end{cases}$$

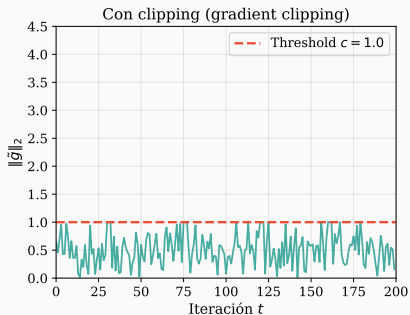
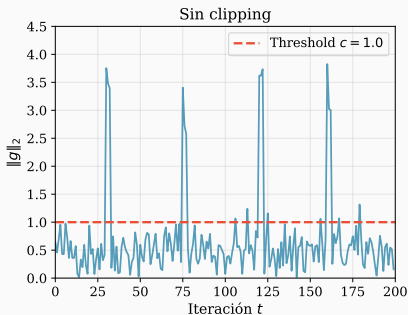


Gradient clipping

Otra heurística importante es **gradient clipping**, que limita la magnitud de los gradientes para estabilizar el entrenamiento:

- **Clip por valor:** $g_i \leftarrow \text{clip}(g_i, -c, c)$.
- **Clip por norma:** si $\|g\| > c$, entonces $g \leftarrow c \cdot \frac{g}{\|g\|}$.

Esto previene el **exploding gradient problem**.



Early stopping

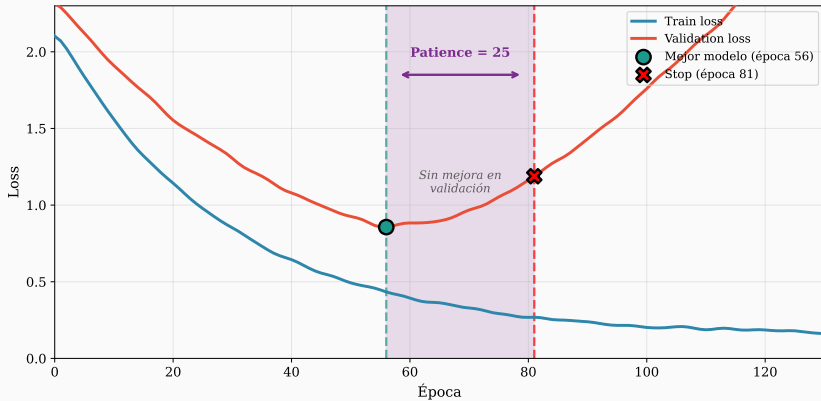
Para evitar el overfitting, se puede usar **early stopping**, que detiene el entrenamiento cuando la evaluación en el conjunto de validación deja de mejorar:

- Se monitorea L_{val} cada $k \in \mathbb{N}$ iteraciones.
- Se guarda el mejor modelo según L_{val} .
- Se detiene si L_{val} no mejora en $P \in \mathbb{N}$ evaluaciones consecutivas.

En el pre-training de LLMs modernos, el overfitting es raro debido al tamaño del corpus.

Early stopping es más relevante en fine-tuning.

Early stopping



Label smoothing

Para mejorar la generalización, se puede usar **label smoothing**, que suaviza las etiquetas one-hot. En lugar de usar $q(y) = \delta_{y^*}(y)$, se define:

$$q_{\text{smooth}}(y) = (1 - \epsilon) \cdot \delta_{y^*}(y) + \frac{\epsilon}{|\mathcal{V}|}$$

donde $\epsilon \in (0, 1)$ es el parámetro de suavizado y $|\mathcal{V}|$ es el tamaño del vocabulario. Esto tiene varios beneficios:

- Previene que el modelo sea sobreconfiado en sus predicciones.
- Regulariza los logits, evitando que crezcan indefinidamente.
- Mejora la generalización a datos no vistos.

Exponential moving average

Los pesos $\theta^{(t)}$ actualizados por el optimizador son ruidosos debido a la aleatoriedad de los mini-batches.

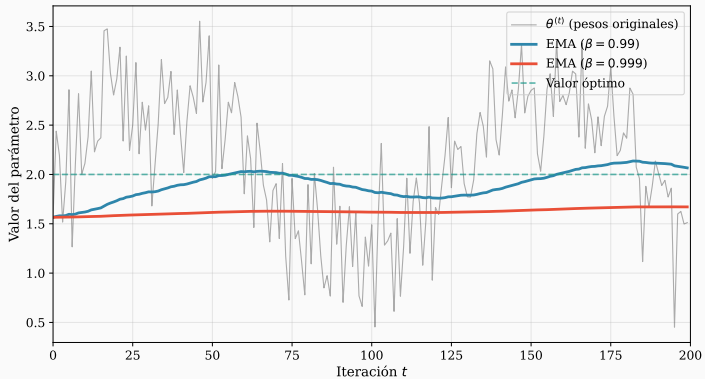
Para obtener una estimación más estable, se puede mantener un **exponential moving average (EMA)** de los pesos:

$$\theta_{\text{EMA}}^{(t)} = \beta \cdot \theta_{\text{EMA}}^{(t-1)} + (1 - \beta) \cdot \theta^{(t)},$$

donde $\beta \in (0.99, 0.9999)$ controla la tasa de decaimiento.

Los pesos EMA suelen generalizar mejor que los pesos finales del optimizador, por lo que se suelen usar para inferencia.

Exponential moving average



Inicialización de pesos

La **inicialización de pesos** permite mantener la varianza de las activaciones estable a través de las capas.

Para una matriz de parámetros $W \in \mathcal{M}_{n_{\text{in}}, n_{\text{out}}}(\mathbb{R})$:

- **Xavier/Glorot (tanh, sigmoide):**

$$W_{ij} \sim \mathcal{U} \left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right)$$

- **He/Kaiming (ReLU):**

$$W_{ij} \sim \mathcal{N} \left(0, \frac{2}{n_{\text{in}}} \right)$$

- **LeCun (SELU):**

$$W_{ij} \sim \mathcal{N} \left(0, \frac{1}{n_{\text{in}}} \right)$$

Mixed precision training

Por otro lado, se puede usar **mixed precision training** para reducir el uso de memoria y acelerar el cómputo:

- **FP16/BF16:** para forward pass y backward pass.
- **FP32:** para acumulación de gradientes y actualización de pesos.

Sin embargo, el uso de baja precisión puede causar underflow en los gradientes. Para mitigar esto, se usan técnicas como:

- **Loss scaling:** escala la pérdida por un factor grande para evitar underflow, y los gradientes se desescalan antes de la actualización.
- **Master weights:** mantiene una copia de los pesos en FP32 para la actualización, pero el forward/backward se hace en FP16.

Large language models

Chat templates

Para construir un modelo conversacional, los LLMs utilizan **chat templates** que estructuran las interacciones.

Usualmente, se definen 3 roles delimitados por tokens especiales. Usando el formato **ChatML** de OpenAI:

- **System:** instrucciones globales y contexto del modelo.
- **User:** mensajes del usuario.
- **Assistant:** respuestas del modelo.

```
<|im_start|>system
Eres un asistente experto en matemáticas.<|im_end|>
<|im_start|>user
¿Cuál es la derivada de  $x^3$ ?<|im_end|>
<|im_start|>assistant
La derivada de  $x^3$  es  $3x^2$ .<|im_end|>
```

Instruction hierarchy

En modelos conversacionales, la **instruction hierarchy** establece un orden de obediencia de las instrucciones:

1. **System prompt:** máxima prioridad, define el comportamiento base y suele mantenerse oculto.
2. **User instructions:** instrucciones del usuario al modelo.
3. **Contenido externo y outputs del modelo:** documentos (RAG), resultados de herramientas, texto generado, etc.

Respetar esta jerarquía ayuda a prevenir ataques como **prompt injection** y requiere entrenamiento específico durante el post-training.

Fine-tuning conversacional

El **fine-tuning conversacional** adapta un modelo de lenguaje pre-entrenado para interacciones multi-turno:

- Se entrena sobre un conjunto de conversaciones con el formato de chat template.
- La pérdida se calcula solo sobre los tokens del **assistant**.
- Se incluyen ejemplos de rechazo de solicitudes.

Sin embargo, esto no es suficiente para alinear completamente el modelo con preferencias humanas.

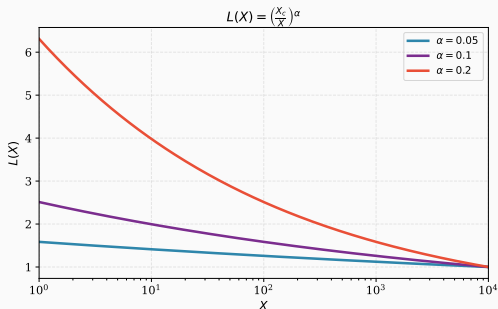
Leyes de escala

Una **ley de escala** es una relación empírica entre la pérdida L y algún recurso X (parámetros, datos, cómputo):

$$L(X) = \left(\frac{X_c}{X} \right)^\alpha + L_\infty,$$

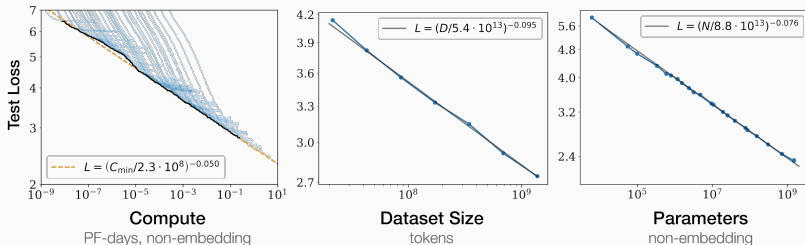
donde X_c es una constante de escala, $\alpha > 0$ es el exponente, y L_∞ es la pérdida irreducible.

- En escala log-log, la relación es lineal.
- Permite extrapolar rendimiento.
- Mayor α implica decaimiento más rápido.



Leyes de escala

En una investigación de OpenAI se estudió cómo escala L respecto al número de parámetros N , la cantidad de datos D y el cómputo C por separado:

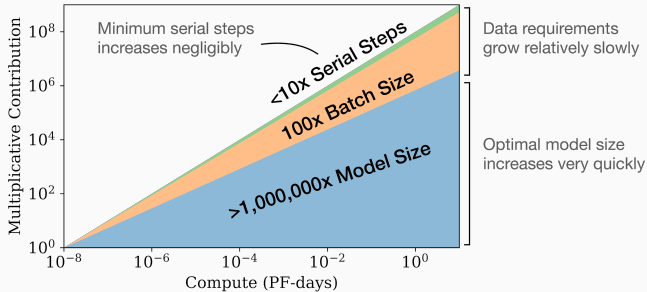


Es decir: $L \propto N^{-0.076}$, $L \propto D^{-0.095}$, $L \propto C^{-0.050}$.

Leyes de escala

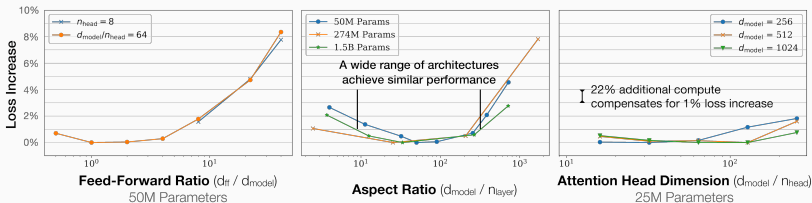
También pudieron concluir cómo asignar eficientemente un presupuesto de cómputo C entre la cantidad de parámetros N y la cantidad de datos D .

Observaron que el tamaño del modelo debe crecer mucho más rápido que la cantidad de datos.



Leyes de escala

También mostraron que la arquitectura específica (FF ratio, aspect ratio, head dimension) tiene poco impacto:



Esto va en línea con la idea de **the bitter lesson** de Sutton.

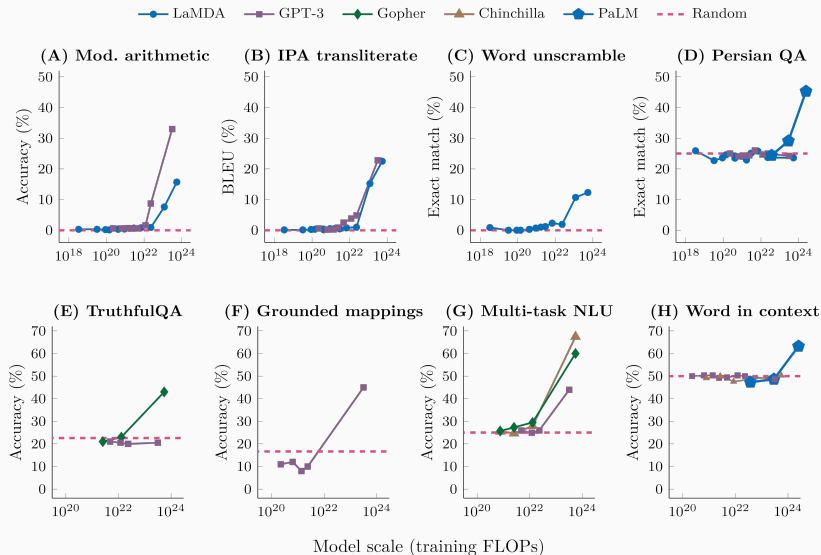
Habilidades emergentes

Una habilidad se dice **emergente** si aparece de manera abrupta al escalar el modelo, pasando de rendimiento aleatorio a alto en un rango estrecho de escala.

Este fenómeno sugeriría la existencia de **transiciones de fase** en las capacidades de los modelos, dificultando predecir qué habilidades aparecerán al escalar.

Un trabajo posterior argumenta que la emergencia es un **artefacto de las métricas discretas**. Al usar métricas continuas, el rendimiento mejora de manera suave y predecible, sin saltos abruptos.

Habilidades emergentes



Alignment y fine-tuning

El **post-training** adapta un modelo pre-entrenado para tareas específicas. El pipeline típico tiene tres etapas:

1. **Pre-training:** next-token prediction sobre corpus masivo.
2. **Supervised Fine-Tuning (SFT):** entrenamiento sobre datos de mejor calidad y formato conversacional.
3. **Alignment:** alineación con preferencias humanas.

Supervised Fine-Tuning

SFT entrena el modelo sobre pares instrucción-respuesta usando la misma pérdida que en el pre-training (máxima verosimilitud), pero solo sobre los tokens de la respuesta.

El dataset de SFT típicamente incluye conversaciones estructuradas, ejemplos de rechazo a solicitudes inapropiadas, y ejemplos de tareas y estilos de respuesta.

Sin embargo, SFT no es suficiente para alignment porque es costoso crear datos que cubran todos los casos y muchas veces es difícil definir explícitamente las preferencias humanas en un dataset o en una función de costo.

Reinforcement Learning from Human Feedback

RLHF alinea el modelo usando preferencias humanas. El proceso tiene tres etapas:

1. **Recolección de preferencias:** humanos comparan pares de respuestas para una misma instrucción.
2. **Entrenamiento de un reward model:** se entrena un modelo para predecir preferencias.
3. **Optimización con RL:** se optimiza el LLM para maximizar el reward usando reward model entrenado.

Por lo tanto, un dataset de preferencias tiene la forma

$\mathcal{D}_{\text{pref}} = \{(x, y_1, y_0)\}$, donde y_1 es la respuesta preferida e y_0 es la respuesta rechazada.

El **reward model** $r_\phi(x, y)$ se entrena para predecir preferencias usando el modelo de Bradley-Terry:

$$p(y_1 \succ y_0 | x) = \sigma(r_\phi(x, y_1) - r_\phi(x, y_0))$$

Este modelo es entrenado mediante máxima verosimilitud:

$$\max_{\phi} \mathbb{E}_{(x, y_1, y_0) \sim \mathcal{D}_{\text{pref}}} [\log \sigma(r_\phi(x, y_1) - r_\phi(x, y_0))]$$

El reward model típicamente se inicializa desde el modelo SFT y reemplaza el cabezal de lenguaje por un cabezal escalar.

Con el reward model r_ϕ entrenado, se optimiza el LLM para maximizar la recompensa regularizada:

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [r_{\phi}(x, y) - \beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})],$$

donde π_{ref} es el modelo SFT y $\beta > 0$ es un hiperparámetro.

Si bien se puede optimizar esta función objetivo directamente (e.g. con REINFORCE), usualmente se utilizan algoritmos más eficientes como **proximal policy optimization (PPO)**.

También es posible optimizar directamente la función de preferencia sin necesidad de un reward model separado utilizando **direct preference optimization (DPO)**.

Low-Rank Adaptation

LoRA es una técnica de **parameter-efficient fine-tuning (PEFT)** que representa las actualizaciones de pesos como productos de matrices de bajo rango.

Para una matriz de pesos $W_0 \in \mathcal{M}_{d,k}(\mathbb{R})$:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}BA$$

donde $B \in \mathcal{M}_{d,r}(\mathbb{R})$, $A \in \mathcal{M}_{r,k}(\mathbb{R})$, $r \ll \min(d, k)$, y α es un hiperparámetro de escala.

W_0 permanece congelado durante el entrenamiento, y solo se entrenan A y B . Esto reduce los parámetros de dk a $r(d + k)$.

En la próxima clase.

- Técnicas de inferencia e inferencia eficiente.
- Prompting y sistemas agénticos.

Modelos Generativos Profundos

Clase 9: Modelos autorregresivos y LLMs (parte V)